Stochastic Block Models

Within networks, nodes can belong to different categories, and these categories can potentially affect the propensity for node interactions. For example, nodes can have different sex categories, and the propensity to interact with nodes of the same sex can be higher than with nodes of different sexes. To model the propensity for interaction between nodes based on the categories they belong to, we can use a stochastic block model approach.

Considerations

> **Caution**
>
> - We consider predefined groups here, with the goal of evaluating the propensity for interaction between nodes within each group.
> - In addition to the block model(s) being tested, we need to include a block where all individuals are considered as belonging to the same group (`Any` in the example). This allows us to assess whether interaction tendencies differ between groups or if the propensity to interact is uniform across all individuals.

Example

Below is an example code snippet demonstrating a Bayesian network model using the stochastic block model approach. This model is a more specific case of Network model, where we assume that nodes are divided into subgroups (blocks). The data is identical to the Network model example, with the addition of covariates *Any*, *Merica*, and *Quantum*, representing the block membership of each node. This example is based on Ross, McElreath, and Redhead (2024).

Python

```
# Setup device-----------------------------------------------
from BayesForge import bf
import jax.numpy as jnp

# Setup device-----------------------------------------------
m = bf(platform='cpu', rand_seed = False)
# Simulate data ----------------------------------------------
N = 50
individual_predictor = m.dist.normal(0,1, shape = (N,1), sample = True)

kinship = m.dist.bernoulli(0.3, shape = (N,N), sample = True)
kinship = kinship.at[jnp.diag_indices(N)].set(0)

category = m.dist.categorical(jnp.array([.25,.25,.25,.25]), sample = True, shape  = (N,))
N_grp, N_by_grp = jnp.unique(category, return_counts=True)
N_grp = N_grp.shape[0]

def sim_network(kinship, individual_predictor,category):
  # Intercept
  B_intercept = m.net.block_model(jnp.full((N,),0), 1, jnp.array([N]), sample = True)
  B_category = m.net.block_model(category, N_grp, N_by_grp, sample = True)

  # SR
  sr = m.net.sender_receiver(
    individual_predictor,
    individual_predictor,
    s_mu = 0.4, r_mu = -0.4, sample = True)

  # D
  DR = m.net.dyadic_effect(kinship, d_sd=2.5, sample = True)



  return m.dist.bernoulli(
    logits = B_intercept + B_category + sr + DR,
    sample = True
    )


network = sim_network(m.net.mat_to_edgl(kinship), individual_predictor, category)

# Predictive model ------------------------------------------------
```

```
m.data_on_model = dict(
    network = network,
    dyadic_predictors = m.net.mat_to_edgl(kinship),
    focal_individual_predictors = individual_predictor,
    target_individual_predictors = individual_predictor,
    category = category
)


def model(network, dyadic_predictors, focal_individual_predictors, target_individual_predicto
    N_id = focal_individual_predictors.shape[0]

    # Block ---------------------------------------
    B_intercept = m.net.block_model(jnp.full((N_id,),0), 1, jnp.array([N_id]), name = "B_inte
    B_category = m.net.block_model(category, N_grp, N_by_grp, name = "B_category")

    ## SR shape =  N individuals---------------------------------------
    sr =  m.net.sender_receiver(
      focal_individual_predictors,
      target_individual_predictors,
      s_mu = 0.4, r_mu = -0.4
    )

    # Dyadic shape = N dyads-----------------------------------
    dr = m.net.dyadic_effect(dyadic_predictors, d_sd=2.5) # Diadic effect intercept only
    m.dist.bernoulli(logits = B_intercept + B_category + sr + dr, obs=network)

m.fit(model, progress_bar=False)
m.summary()
```

```
An NVIDIA GPU may be present on this machine, but a CUDA-enabled jaxlib is not installed. Fal

bf v 0.0.60 package loaded
jax.local_device_count 16
```

	mean	sd	eti89_lb	eti89_ub	ess_bulk	ess_tail	r_hat	mcse_mean	mcse_s
b_B_category[0, 0]	-2.32	1.42	-4.56	-0.05	3977.16	3196.43	1.0	0.02	0.02
b_B_category[0, 1]	-7.62	1.92	-10.88	-4.64	5819.86	2842.33	1.0	0.03	0.03
b_B_category[0, 2]	-6.93	2.04	-10.40	-3.81	6359.01	3096.07	1.0	0.03	0.03

	mean	sd	eti89_lb	eti89_ub	ess_bulk	ess_tail	r_hat	mcse_mean	mcse_s
b_B_category[0, 3]	-6.86	2.03	-10.24	-3.74	6162.80	3312.56	1.0	0.03	0.03
b_B_category[1, 0]	-4.36	1.27	-6.45	-2.39	3498.56	3088.68	1.0	0.02	0.02
...	...	...	...	...	...	...	...	...	...
sr_rf[48, 1]	0.18	0.83	-0.98	1.64	4585.96	2802.03	1.0	0.01	0.02
sr_rf[49, 0]	-1.55	2.49	-5.94	2.04	5008.48	3182.78	1.0	0.04	0.04
sr_rf[49, 1]	0.18	0.76	-0.90	1.51	4806.42	2688.32	1.0	0.01	0.02
sr_sigma[0]	3.00	0.79	1.86	4.34	1673.44	1923.23	1.0	0.02	0.01
sr_sigma[1]	0.76	0.55	0.07	1.74	1077.03	1433.94	1.0	0.02	0.01

R

```r
library(BayesForge)

# Setup device-------------------------------------------------
m <- importBF(platform = "cpu", rand_seed = FALSE)
jnp <- reticulate::import("jax.numpy")

# Simulate data -----------------------------------------------
N <- 50L
individual_predictor <- m$dist$normal(0, 1, shape = reticulate::tuple(N, 1L), sample = TRUE)

kinship <- m$dist$bernoulli(0.3, shape = reticulate::tuple(N, N), sample = TRUE)
diag_idx <- jnp$diag_indices(N)
kinship <- kinship$at[reticulate::tuple(diag_idx[[1]], diag_idx[[2]])]$set(0L)

category <- m$dist$categorical(jnp$array(c(0.25, 0.25, 0.25, 0.25)), sample = TRUE, shape = 
u <- jnp$unique(category, return_counts = TRUE)
N_grp <- u[[1]]$shape[[1]]
N_by_grp <- u[[2]]

sim_network <- function(kinship, individual_predictor, category) {
  # Intercept
  B_intercept <- m$net$block_model(jnp$full(reticulate::tuple(N), 0L), 1L, jnp$array(list(N)))
  B_category <- m$net$block_model(category, N_grp, N_by_grp, sample = TRUE)

  # SR
  sr <- m$net$sender_receiver(
    individual_predictor,
    individual_predictor,
```

```
    s_mu = 0.4, r_mu = -0.4, sample = TRUE
  )

  # D
  DR <- m$net$dyadic_effect(kinship, d_sd = 2.5, sample = TRUE)

  m$dist$bernoulli(
    logits = B_intercept + B_category + sr + DR,
    sample = TRUE
  )
}

network <- sim_network(m$net$mat_to_edgl(kinship), individual_predictor, category)

# Predictive model ------------------------------------------------

m$data_on_model <- list(
  network = network,
  dyadic_predictors = m$net$mat_to_edgl(kinship),
  focal_individual_predictors = individual_predictor,
  target_individual_predictors = individual_predictor,
  category = category
)

model <- function(network, dyadic_predictors, focal_individual_predictors, target_individual
  N_id <- focal_individual_predictors$shape[[1]]

  # Block ----------------------------------------
  B_intercept <- m$net$block_model(jnp$full(reticulate::tuple(N_id), 0L), 1L, jnp$array(list
  B_category <- m$net$block_model(category, N_grp, N_by_grp, name = "B_category")

  ## SR shape =  N individuals----------------------------------------
  sr <- m$net$sender_receiver(
    focal_individual_predictors,
    target_individual_predictors,
    s_mu = 0.4, r_mu = -0.4
  )

  # Dyadic shape = N dyads----------------------------------------
  dr <- m$net$dyadic_effect(dyadic_predictors, d_sd = 2.5) # Dyadic effect intercept only
  m$dist$bernoulli(logits = B_intercept + B_category + sr + dr, obs = network, name = "y")
}
```

```
m$fit(model, progress_bar = FALSE)
m$summary()
```

Julia

```
# Setup device-----------------------------------------------
using PyBayesForge

# Setup device-----------------------------------------------
m = importBF(platform="cpu", rand_seed = false)

# Simulate data ---------------------------------------------
N = 50
individual_predictor = m.dist.normal(0,1, shape = (N,1), sample = true)

kinship = m.dist.bernoulli(0.3, shape = (N,N), sample = true)
kinship = kinship.at[jnp.diag_indices(N)].set(0)

category = m.dist.categorical(jnp.array([.25,.25,.25,.25]), sample = true, shape  = (N,))
N_grp, N_by_grp = jnp.unique(category, return_counts=true)
N_grp = N_grp.shape[0]


function sim_network(kinship, individual_predictor, category)
  # Intercept
  B_intercept = m.net.block_model(jnp.full((N,),0), 1, jnp.array((N,)), sample = true)

  # Block
  B_category = m.net.block_model(category, N_grp, N_by_grp, sample = true)

  # SR
  sr = m.net.sender_receiver(individual_predictor, individual_predictor, s_mu = 0.4, r_mu =

  # D
  DR = m.net.dyadic_effect(kinship, d_sd=2.5, sample = true)

  return m.dist.bernoulli(logits = B_intercept + B_category + sr + DR, sample = true)

end
```

```
network = sim_network(m.net.mat_to_edgl(kinship), individual_predictor, category)


# Predictive model ------------------------------------------------

m.data_on_model = pydict(
    network = network,
    dyadic_predictors = m.net.mat_to_edgl(kinship),
    focal_individual_predictors = individual_predictor,
    target_individual_predictors = individual_predictor,
    category = category
)


@BF function model(network, dyadic_predictors, focal_individual_predictors, target_individual
    N_id = focal_individual_predictors.shape[0]

    # Block ---------------------------------------
    B_intercept = m.net.block_model(jnp.full((N_id,),0), 1, jnp.array((N_id,)), name = "B_int
    B_category = m.net.block_model(category, N_grp, N_by_grp, name = "B_category")

    ## SR shape =  N individuals---------------------------------------
    sr =  m.net.sender_receiver(
      focal_individual_predictors,
      target_individual_predictors,
      s_mu = 0.4, r_mu = -0.4
    )

    # Dyadic shape = N dyads---------------------------------------
    dr = m.net.dyadic_effect(dyadic_predictors, d_sd=2.5) # Diadic effect intercept only
    m.dist.bernoulli(logits = B_intercept + B_category + sr + dr, obs=network)
end

m.fit(model, num_samples = 500, num_warmup = 500, num_chains = 1)
m.summary()
```

Mathematical Details

Main Formula

The model's block structure can be represented by the following formula for binary ties (or analogously with a Poisson distribution and log link for weighted counts). Note that the sender-receiver and dyadic effects are not represented here, as they are already accounted for in the Network model chapter:

$$G_{ij} \sim \text{Bernoulli}(p_{ij})$$

$$\text{logit}(p_{ij}) = B_{k(i),k(j)}$$

where:

- B is a matrix of intercept parameters unique to the interaction of categories. For example, if there are three groups, then B will be a 3x3 matrix where each element gives the log-odds (or log-rate) of an individual in group k interacting with an individual in group l.
- We use the function k, to return the group identity (i.e., the block) of individual i.

Defining formula sub-equations and prior distributions

To account for all link rates between categories, we can define a square matrix B as follows: the off-diagonal elements represent the link rates between categories i and j, while the diagonal elements represent the link rates within category i.

$$B_{i,j} = \begin{bmatrix} a_{1,1} & a_{1,2} & \cdots & a_{1,j} \\ a_{2,1} & a_{2,2} & \cdots & a_{2,j} \\ \vdots & \vdots & \ddots & \vdots \\ a_{i,1} & a_{i,2} & \cdots & a_{i,j} \end{bmatrix}$$

As we consider the link probability within categories to be higher than the link probabilities between categories, we define different priors for the diagonal and the off-diagonal. Priors should also depend on sample size, N, so that the resultant network density approximates empirical networks. Basic priors could be:

$$a_{k \to k} \sim \text{Normal}\left(\text{Logit}\left(\frac{0.1}{\sqrt{N_k}}\right), 1.5\right)$$

$$a_{k \to \tilde{k}} \sim \text{Normal}\left(\text{Logit}\left(\frac{0.01}{0.5\sqrt{N_k} + 0.5\sqrt{N_{\tilde{k}}}}\right), 1.5\right)$$

where:

- $k \to k$ indicates a diagonal element.
- $k \to \tilde{k}$ indicates an off-diagonal element.

Model validation tests

- Comparison with Stan: Social Relations Model fitted with BayesForge and Stan: fitting time and posterior distributions compared.
- Documentation examples: the Python, R and Julia examples of this page are run by the documentation tests (`Python/test_model_pages.py`, `R/test_r_model_pages.py`, `Julia/test_julia_model_pages.py`).

Note(s)

> **Note**
>
> - By defining this block model within our network model, we are estimating assortativity and disassortativity for categorical variables.
> - Similarly, for continuous variables, we can generate a block model that includes all continuous variables.

Reference(s)

Ross, Cody T, Richard McElreath, and Daniel Redhead. 2024. "Modelling Animal Network Data in r Using STRAND." *Journal of Animal Ecology* 93 (3): 254–66.